

CORS & Database — NutriGuide Explained



PART 1 — What is CORS and What Does It Do in Our Project?

🤔 The Problem CORS Solves

Imagine two people living at **different addresses** trying to share a secret document.

Your **React frontend** runs at → `http://localhost:5173`

Your **Express backend** runs at → `http://localhost:5000`

These are **two different "origins"** (different ports = different origins).

By default, browsers have a built-in security rule called the **Same-Origin Policy**:

❌ "You are NOT allowed to fetch data from a different address/port than the one your page loaded from."

So when your React app tries to call `http://localhost:5000/api/auth/login` ... the **browser blocks it!**

React App (port 5173) — fetch("http://localhost:5000/api/login")
→ Backend (port 5000)

← Response

Browser intercepts ← ❌ BLOCKED! "Cross-Origin Request Blocked"

This is where **CORS** (Cross-Origin Resource Sharing) comes in.

✅ What CORS Does — It's Permission Given by the Server

CORS is NOT a frontend thing. It's configured on the BACKEND.

The backend tells the browser: **"Hey, I trust requests from this address — let them through."**

The backend does this by adding special **HTTP headers** to every response:

```
Access-Control-Allow-Origin: *  
Access-Control-Allow-Methods: GET, POST, PUT, DELETE, OPTIONS  
Access-Control-Allow-Headers: Content-Type, Authorization
```

When the browser sees these headers, it says:  "Okay, the server allows this — I'll let the response through."

CORS in Our NutriGuide Project — `server.js`

```
app.use(cors({  
  origin: '*',  
  methods: ['GET', 'POST', 'PUT', 'DELETE', 'OPTIONS'],  
  allowedHeaders: ['Content-Type', 'Authorization']  
}));
```

Let's break down each line:

Config	Value	What it means
<code>origin</code>	<code>'*'</code>	Any frontend URL can talk to this backend (the <code>*</code> is a wildcard = "everyone")
<code>methods</code>	<code>GET, POST, PUT, DELETE, OPTIONS</code>	These are the only HTTP actions the frontend is allowed to use
<code>allowedHeaders</code>	<code>Content-Type, Authorization</code>	The request can only send these two headers

`Content-Type` → tells the backend "I'm sending JSON data"

`Authorization` → carries the JWT token: `"Bearer eyJ..."`

How CORS Works Step by Step in Our App

Simple Request (e.g., Login)

1. React app sends POST `http://localhost:5000/api/auth/login` with header: `Content-Type: application/json`
2. Express backend processes the request
3. Before sending the response, the `cors()` middleware automatically adds these headers to the response:
`Access-Control-Allow-Origin: *`
`Access-Control-Allow-Methods: GET, POST, PUT, DELETE, OPTIONS`
4. Browser sees `"origin: *"` →  allows the response to reach React

Preflight Request (e.g., PUT profile — with custom Authorization header)

For certain requests (like PUT with Authorization header), the browser first sends a **"preflight" check** using the `OPTIONS` method — like knocking on the door before entering.

1. Browser → `OPTIONS /api/auth/profile`
"Hey server, can I send a PUT request with an Authorization header?"
2. Backend (cors middleware) → responds:
`Access-Control-Allow-Methods: GET, POST, PUT, DELETE, OPTIONS` 
`Access-Control-Allow-Headers: Content-Type, Authorization` 
3. Browser → "Great, server allows it!" → sends the actual PUT request
4. Backend processes it and responds normally

What Would Happen Without CORS?

```
// If server.js had NO cors() middleware...

app.use(express.json()); // just this, no cors

// Then when React calls /api/auth/login:
// ✗ Browser shows: "Access to fetch at 'http://localhost:5000'
//   from origin 'http://localhost:5173' has been blocked by CORS
//   policy"
// The app would completely break – no login, no data, nothing works!
```

In Production (Real Deployed App)

Right now `origin: '*'` means **anyone** can call your API. This is fine for development, but in production you should restrict it:

```
// More secure for production:
app.use(cors({
  origin: 'https://your-nutriguide-app.com', // only your real domain
  methods: ['GET', 'POST', 'PUT', 'DELETE'],
  allowedHeaders: ['Content-Type', 'Authorization']
}));
```

PART 2 — The NutriGuide Database Explained

What Database Do We Use?

MySQL — a popular, structured database that stores data in **tables** (like Excel sheets with rows and columns).

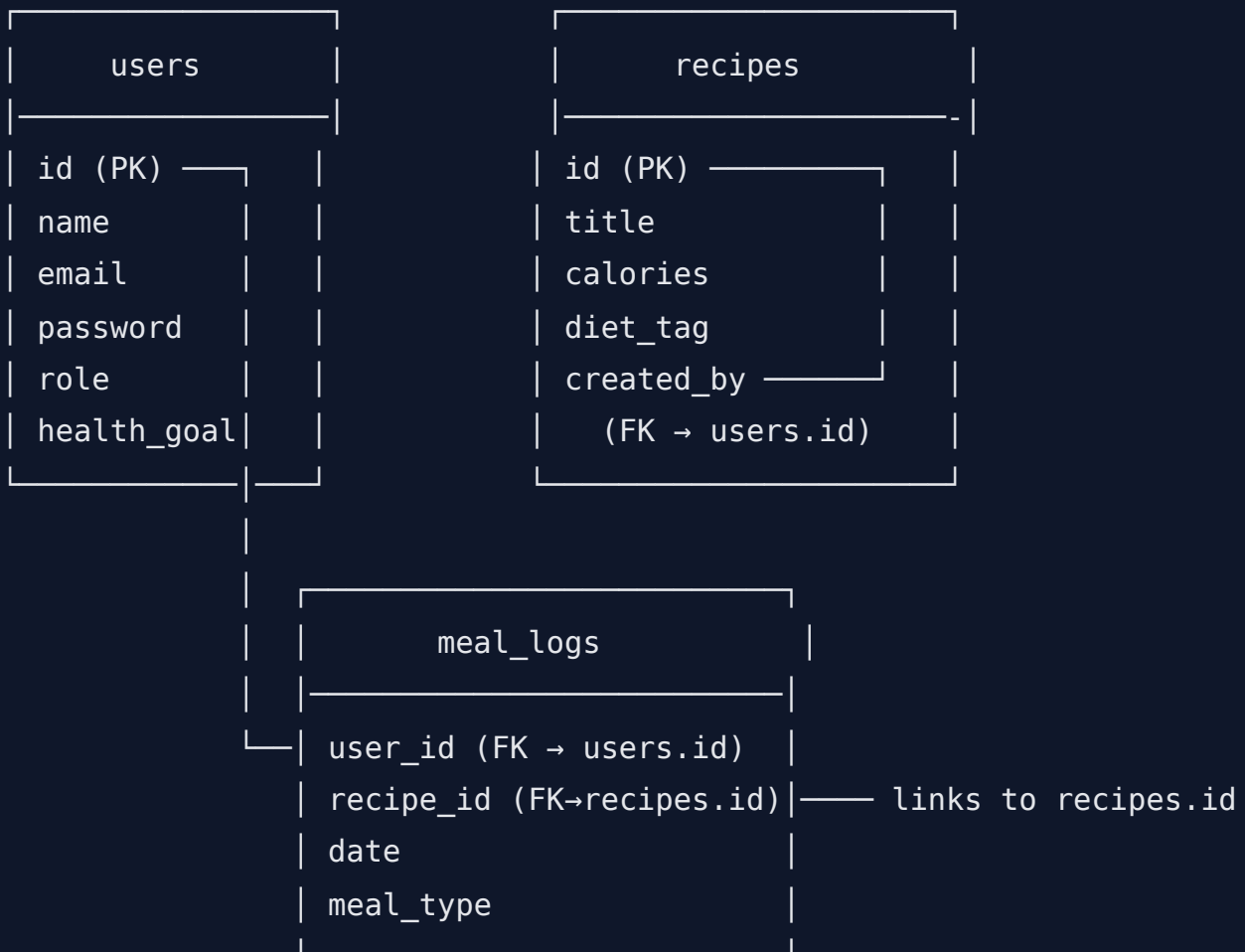
The database is named: `nutriguide`

It has **3 tables**:

nutriguide (database)

└─ users ← who uses the app
└─ recipes ← all Indian meal recipes
└─ meal_logs ← which user ate what, when

How The 3 Tables Are Related



- **PK** = Primary Key (unique identifier for each row)
- **FK** = Foreign Key (a reference/link to another table)

Table 1: users

This table stores every person who registers on NutriGuide.

```
CREATE TABLE users (
  id          INT AUTO_INCREMENT PRIMARY KEY,
  name        VARCHAR(255) NOT NULL,
  email       VARCHAR(255) UNIQUE NOT NULL,
  password    VARCHAR(255) NOT NULL,
  role        ENUM('user', 'admin') DEFAULT 'user',
  age         INT,
  weight      FLOAT,
  height      FLOAT,
  diet_preference ENUM('none', 'vegan', 'vegetarian', 'non-vegetarian') DEFAULT 'none',
  health_goal  ENUM('lose_weight', 'maintain', 'gain_muscle')
  DEFAULT 'maintain',
  created_at   TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```

What Each Column Means

Column	Type	Example	Notes
id	INT	1, 2, 3 ...	Auto-increases, never repeat
name	VARCHAR(255)	"Moraya"	Max 255 characters
email	VARCHAR(255)	"moraya@gmail.com"	Must be unique — no two users same email
password	VARCHAR(255)	"\$2b\$10\$xyz..."	Hashed by bcrypt, never plain text
role	ENUM	"user" or "admin"	Controls what you can access
age	INT	22	Can be empty (NULL)
weight	FLOAT	65.5	In kg, decimal allowed
height	FLOAT	170.0	In cm

diet_preference	ENUM	"vegetarian"	Only these 4 values allowed
health_goal	ENUM	"lose_weight"	Only these 3 values allowed
created_at	TIMESTAMP	2026-05-09 10:00:00	Auto-set when user registers

Real Example Row

```
id=1, name="Moraya", email="moraya@gmail.com",
password="$2b$10$Kh...(hashed)", role="user",
age=22, weight=70, height=165,
diet_preference="vegetarian", health_goal="lose_weight"
```

Table 2: recipes

This table stores all the Indian meal recipes shown in the app.

```
CREATE TABLE recipes (
  id          INT AUTO_INCREMENT PRIMARY KEY,
  title       VARCHAR(255) NOT NULL,
  description TEXT NOT NULL,
  ingredients JSON NOT NULL,
  instructions TEXT NOT NULL,
  calories    INT NOT NULL,
  protein     INT NOT NULL,
  carbs       INT NOT NULL,
  fat         INT NOT NULL,
  diet_tag    VARCHAR(50),
  image_url   VARCHAR(500),
  created_by  INT,
  created_at  TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  FOREIGN KEY (created_by) REFERENCES users(id) ON DELETE SET NULL
);
```

What Each Column Means

Column	Type	Example	Notes
id	INT	1, 2 ...	Unique recipe ID
title	VARCHAR(255)	"Dal Tadka"	Recipe name
description	TEXT	"A classic lentil dish..."	Long description
ingredients	JSON	["dal", "ghee", "turmeric"]	Array stored as JSON
instructions	TEXT	"Step 1: Boil dal..."	Cooking steps
calories	INT	320	Total calories
protein	INT	15	Grams of protein
carbs	INT	45	Grams of carbohydrates
fat	INT	8	Grams of fat
diet_tag	VARCHAR(50)	"vegetarian"	Used for recommendation filtering
image_url	VARCHAR(500)	"https://..."	Link to dish photo
created_by	INT (FK)	1	Which admin user created this recipe

JSON column — MySQL can store a full JSON array in one column: ["dal", "ghee", "turmeric", "onion"]

When the backend reads it, it uses `JSON.parse()` to turn it back into a JavaScript array.

Real Example Row

```
id=5, title="Dal Tadka", description="Classic lentil dish",
ingredients='["toor dal","ghee","cumin","garlic"]',
calories=320, protein=15, carbs=45, fat=8,
diet_tag="vegetarian", image_url="https://cdn.../dal.jpg",
created_by=1
```

Table 3: meal_logs

This is the **activity log** — it records every meal a user eats. It connects users ↔ recipes.

```
CREATE TABLE meal_logs (
  id          INT AUTO_INCREMENT PRIMARY KEY,
  user_id     INT NOT NULL,
  recipe_id   INT NOT NULL,
  date        DATE NOT NULL,
  meal_type   ENUM('breakfast', 'lunch', 'dinner', 'snack') NOT NULL,
  created_at  TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  FOREIGN KEY (user_id)   REFERENCES users(id)   ON DELETE CASCADE,
  FOREIGN KEY (recipe_id) REFERENCES recipes(id) ON DELETE CASCADE
);
```

What Each Column Means

Column	Type	Example	Notes
id	INT	1, 2 ...	Unique log ID
user_id	INT (FK)	1	Links to users.id — "who ate it?"
recipe_id	INT (FK)	5	Links to recipes.id — "what did they eat?"
date	DATE	2026-05-09	"when did they eat it?"
meal_type	ENUM	"lunch"	Only 4 options allowed

Real Example Rows

```
id=1, user_id=1, recipe_id=5, date="2026-05-09", meal_type="breakfast"
id=2, user_id=1, recipe_id=12, date="2026-05-09", meal_type="lunch"
id=3, user_id=1, recipe_id=3, date="2026-05-09", meal_type="dinner"
```

This means: User 1 (Moraya) ate recipe 5 for breakfast, recipe 12 for lunch, and recipe 3 for dinner on May 9, 2026.

What are Foreign Keys? (Simple Explanation)

Foreign Keys are **links between tables** that ensure data stays consistent.

```
ON DELETE CASCADE → if a user is deleted, all their meal_logs are also
deleted automatically
ON DELETE SET NULL → if a user (admin) who created a recipe is deleted,
the recipe stays
                        but created_by is set to NULL
```

Example of why this matters:

```
Without FK: You could log meal_log { user_id: 999, recipe_id: 888 }
            Even if user 999 and recipe 888 don't exist! → Corrupt data
```



```
With FK:    MySQL automatically rejects this insert
            "user_id 999 does not exist in users table" ❌ → Data stays
clean ✅
```

How the Backend Queries the Database — Real Examples

Get today's meals for a user

```
SELECT m.id, m.date, m.meal_type, r.title, r.calories, r.protein,
r.carbs, r.fat
FROM meal_logs m
JOIN recipes r ON m.recipe_id = r.id
WHERE m.user_id = 1 AND m.date = '2026-05-09'
```

JOIN combines data from two tables in one query.

This gets the meal log AND the recipe info (title, calories etc.) in one go.

Get personalized recipe recommendations

```
-- Step 1: Get user's diet preference
SELECT diet_preference, health_goal FROM users WHERE id = 1;
-- Result: { diet_preference: 'vegetarian', health_goal: 'lose_weight' }

-- Step 2: Get matching recipes
SELECT * FROM recipes WHERE diet_tag = 'vegetarian';

-- Step 3 (in JS): Filter out allergens, sort by calories ascending
```

Get weekly nutrition analytics

```
SELECT m.date,
       SUM(r.calories) as total_calories,
       SUM(r.protein) as total_protein,
       SUM(r.carbs) as total_carbs,
       SUM(r.fat) as total_fat
FROM meal_logs m
JOIN recipes r ON m.recipe_id = r.id
WHERE m.user_id = 1
      AND m.date >= DATE_SUB(CURDATE(), INTERVAL 7 DAY)
GROUP BY m.date
ORDER BY m.date ASC
```

This aggregates all meals per day to get the **daily total nutrition** — used for the Analytics chart (Recharts).



Summary — Database at a Glance

